International Journal of Scientific Research in Computer Science, Engineering and Information Technology
Peer Reviewed and Refereed International Scientific Research Journal
ISSN : 2456-3307
Available Online at : ijsrcseit.com
doi : https://doi.org/10.32628/CSEIT2612335

Zero-Trust Identity for Autonomous AI Agents: Unifying SPIFFE, OAuth 2.1 with PKCE, OIDC, and OPA in Multi-Agent Architectures

Venkata Krishna Prasanth Budigi[1], Maheshbabu Dhanekula[2], Siri Chandana Sirigiri[3], Srinivas Rao Marri[4]

[1]University of Dayton, Ohio, USA
[2]University of Central Missouri, USA
[3]Wright State University, USA
[4]Independent Researcher, USA

ARTICLEINFO

Article History:

Accepted : 30 April 2026
Published: 23 May 2026

Publication Issue
Volume 12, Issue 3
May-June-2026

Page Number
328-340

ABSTRACT

The rapid adoption of autonomous AI agents in distributed microservices architectures introduces a critical and largely unsolved security challenge: how to establish, verify, and enforce identity across every layer of an agentic workflow. In traditional systems, identity management addresses two well-understood dimensions: human user identity (authenticated via passwords, single sign-on, or multi-factor mechanisms) and a limited number of service identities (managed through static API keys or long-lived service accounts). These approaches suffice when services execute predetermined logic on behalf of known callers. However, the emergence of agentic workflows, where autonomous agents reason, invoke tools, and chain multi-step actions across service boundaries, fundamentally changes the identity landscape. Each agent workload requires its own cryptographically verifiable workload identity, and the system must maintain governed identity propagation that securely carries both the originating user's authorization context and the calling service's identity through every hop of the request chain, all while preserving secure session state across process boundaries and service restarts.

Moreover, just as APIs are classified into public, protected, and private tiers with progressively stricter access controls, agents must be similarly classified: public agents accessible to any authenticated user, protected agents restricted to users with specific roles, and private agents accessible only through authorized internal services with both valid user credentials and verified workload identity.

We present AgentSecurity, a production-ready framework that unifies three complementary standards to achieve zero-trust identity with governed identity

Copyright © 2026 The Author(s) : This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/)

propagation for multi-agent systems: (1) OAuth 2.1 with Proof Key for Code Exchange (PKCE) for secure human authentication via OpenID Connect (OIDC), with disk-persisted session state that survives process restarts, eliminating authorization code interception attacks; (2) SPIFFE JWT-SVIDs (JSON Web Token-based SPIFFE Verifiable Identity Documents) for cryptographic workload identity, replacing heavyweight X.509 mutual TLS (mTLS) with lightweight, audience-scoped, short-lived JWTs that give every agent its own verifiable identity; and (3) the Open Policy Agent (OPA) for declarative, two-dimensional authorization that jointly evaluates user roles and caller service identity in a single policy decision, ensuring that identity propagation is governed at every trust boundary.

We demonstrate this framework on a real multi-agent platform comprising LangGraph agents orchestrated via the Model Context Protocol (MCP), where every inter-service call carries both a user identity token and a workload identity token, propagating the user's authorization context through the entire agentic pipeline. Every access decision is governed by auditable, version-controlled Rego policies. Our evaluation shows that the framework adds less than 12 ms of latency overhead per request while providing four independent authentication layers, fail-closed authorization, and automatic credential rotation. A complete reference implementation is available at https://github.com/krishna1501/AgentSecurity.

Keywords: Zero-trust security, AI agents, SPIFFE, OAuth 2.1, PKCE, OIDC, Open Policy Agent, multi-agent systems, workload identity, JWT-SVID, LangGraph, Model Context Protocol

Introduction

Large Language Model (LLM)-powered agents are rapidly moving from research prototypes to production deployments [1, 2]. Unlike traditional API services, these agents possess autonomy: they can reason over user requests, invoke external tools, and chain multi-step actions without human intervention. When deployed as distributed microservices, this autonomy creates a security surface that existing identity and access management (IAM) frameworks were not designed to address.

Consider a multi-agent platform where a user asks "What is Apple's stock price?" The request traverses multiple service boundaries: a frontend authenticates the user, an orchestrator classifies intent and routes the request, and a specialized stock agent invokes financial data tools. At each boundary, two questions must be answered simultaneously:

1. **User identity:** Is the human behind this request authenticated and authorized to access stock data?
2. **Workload identity:** Is the calling service actually the legitimate orchestrator, or a compromised component attempting lateral movement?

Answering only one question is insufficient. A stolen user token without workload verification enables confused-deputy attacks. A valid workload identity without user authorization enables privilege

escalation. The security model must verify both dimensions simultaneously at every trust boundary.

1.1 Contributions

This paper makes the following contributions:

1. **JWT-SVIDs as lightweight workload identity:** We replace X.509 certificate-based mTLS with SPIFFE JWT-SVIDs, which are short-lived (1-hour TTL), audience-scoped JWTs signed by a SPIRE certificate authority. This eliminates PKI infrastructure complexity while providing cryptographically verifiable service identity via a simple HTTP header (Section 6).
2. **Two-dimensional OPA authorization:** We introduce a Rego policy model that evaluates both user roles (from OIDC claims) and caller SPIFFE IDs in a single, atomic policy decision. This ensures that access requires both user authorization and service authentication (Section 7).
3. **PKCE-secured OAuth 2.1 with disk-persisted state:** We implement the Authorization Code flow with Proof Key for Code Exchange (PKCE) using the S256 challenge method, with a novel disk-persistence mechanism that survives process restarts in reactive UI frameworks (Section 5).
4. **Agent classification tiers:** Analogous to public, protected, and private API classifications, we define a three-tier agent classification model where each tier enforces progressively stricter identity controls, from simple user authentication through full dual-dimensional workload identity verification (Section 4.3).
5. **End-to-end reference architecture:** We demonstrate these techniques in a production-grade platform with LangGraph agents, MCP orchestration, and four independent authentication layers (Section 4).

Background and Related Work

2.1 Identity in Distributed Systems

Service-to-service authentication has traditionally relied on shared secrets, API keys, or mTLS with X.509 certificates. The Secure Production Identity Framework for Everyone (SPIFFE) specification [3] addresses limitations of these approaches by providing a standardized, platform-agnostic workload identity framework. The SPIFFE Runtime Environment (SPIRE) [4] implements SPIFFE by issuing short-lived identity documents, in the form of either X.509 SVIDs or JWT-SVIDs, to workloads based on attestation.

While SPIFFE/SPIRE have gained adoption in Kubernetes-native infrastructure [5], their application to AI agent architectures remains unexplored. Existing SPIFFE deployments focus on infrastructure services (databases, message queues) rather than autonomous agents that make decisions on behalf of users.

2.2 OAuth 2.1 and PKCE

OAuth 2.1 [6] consolidates best practices from OAuth 2.0, mandating PKCE [7] for all authorization code grants. PKCE prevents authorization code interception attacks by binding the token exchange to a cryptographic verifier generated by the client. The S256 challenge method computes:

$$challenge = BASE64URL(SHA256(verifier)) \quad (1)$$

The authorization server verifies that the client presenting the authorization code also possesses the original verifier, ensuring that intercepted codes cannot be exchanged by attackers.

2.3 Policy-as-Code Authorization

Open Policy Agent (OPA) [8] enables declarative authorization through the Rego policy language. Prior work has applied OPA to Kubernetes admission control [9] and API gateway authorization, but not to the specific challenge of jointly evaluating human identity and workload identity for AI agent systems.

2.4 AI Agent Security

Recent surveys [10, 11] identify authentication and authorization as critical gaps in agent security. Most multi-agent frameworks (AutoGen [12], CrewAI, LangGraph [13]) provide no built-in identity or access control mechanisms, delegating security to the deployment environment. Our work directly

addresses this gap with a portable, standards-based solution.

Threat Model

We consider an adversary with the following capabilities:

- **T1 (Token theft):** The adversary obtains a valid user JWT, for example via Cross-Site Scripting (XSS) or log exposure.
- **T2 (Authorization code interception):** The adversary intercepts an OAuth authorization code during the redirect flow.
- **T3 (Compromised service):** One microservice is compromised and attempts to call other agent APIs directly (lateral movement).
- **T4 (Confused deputy):** A legitimate service is tricked into making unauthorized requests on behalf of a user without proper role verification.
- **T5 (Policy bypass):** The adversary attempts to access agents by exploiting gaps between authentication and authorization logic.

Table 1: Threat-to-Mitigation Mapping Across the Four Authentication Layers

Threat	Mitigation	Layer
T1: Token theft	Short-lived JWTs with audience binding	L1, L3
T2: Code intercept	PKCE S256 challenge-response	L1
T3: Lateral movement	SPIFFE caller ID verification	L3, L4
T4: Confused deputy	Dual-dimensional OPA policy	L2, L4
T5: Policy bypass	Fail-closed OPA with dual validation	L2, L4

Architecture

4.1 System Overview

AgentSecurity implements a layered microservices architecture where each component operates as an independent service with its own authentication boundary. The system comprises six components:

1. **Streamlit UI (port 8501).** For demonstration purposes, we use Streamlit as the frontend to implement the user-facing OAuth 2.1 Authorization Code + PKCE flow, and Auth0 as the OIDC provider. The framework is agnostic to both: any frontend (React, Angular, mobile) can implement the PKCE flow, and any OAuth 2.1-compliant identity provider (Okta, Keycloak, Azure AD, Google Identity) can replace Auth0.
2. **MCP Server (port 8001).** Orchestration gateway implementing the Model Context Protocol [14]. Classifies user intent using Claude Haiku, checks authorization via OPA, and routes to the appropriate agent API with both user and service identity tokens.
3. **Weather API (port 8002).** FastAPI microservice hosting a LangGraph weather agent with three parallel data sources.
4. **Stock API (port 8003).** FastAPI microservice hosting a LangGraph stock agent with financial data tools.
5. **OPA Server (port 8181).** Evaluates Rego authorization policies, with a local Python fallback when unavailable.
6. **SPIRE Server (port 8081).** Certificate authority issuing JWT-SVIDs with a 1-hour TTL, auto-refreshed every 30 minutes.

4.2 Four Authentication Layers

Every request traverses four independent authentication layers, as summarized in Table 2. Critically, both the MCP server and downstream agent APIs independently validate all applicable layers.

Table 2: Four Authentication Layers Enforced at Each Trust Boundary

Layer	Purpose	Mechanism
L1	User Identity	Auth0 JWT (RS256, audience + issuer check)
L2	User Authorization	Role claims evaluated via OPA policy
L3	Service Identity	SPIFFE JWT-SVID in X-SPIFFE-JWT header
L4	Service Authorization	OPA allowed_callers rule

4.3 Agent Classification: Public, Protected, and Private

Just as REST APIs are classified into public, protected, and private tiers with progressively stricter access controls, AI agents must be similarly classified to enforce appropriate identity security at each tier. This classification determines which authentication layers are enforced and how identity propagation is governed.

Table 3: Agent Classification Tiers with Corresponding Identity and Access Control Requirements

Tier	Auth Layers	Access Policy
Public	L1 only (user JWT)	Any authenticated user; no role or service identity checks
Protected	L1 + L2 (user JWT + role)	User must hold a specific role (e.g., weather-user); no service identity required
Private	L1 + L2 + L3 + L4 (full stack)	User must hold a valid role and the caller must present a verified SPIFFE JWT-SVID from an authorized service

Public agents serve general-purpose functions such as health checks or publicly available information lookups. Protected agents handle domain-specific tasks that require explicit role-based authorization. Private agents operate at the highest security tier and enforce the full four-layer authentication stack.

The OPA policy naturally supports this tiered model. The role_to_spiffe mapping governs which roles can reach which agents (separating public from protected), while the allowed_callers mapping governs which services may invoke each agent (separating protected from private). Promoting an agent from public to protected requires adding a role constraint in the policy. Promoting from protected to private requires adding a caller restriction. Both changes are declarative, auditable, and require no code modifications to the agent itself.

OAuth 2.1 with PKCE Implementation

5.1 Authorization Code Flow with S256 Challenge

The frontend implements the OAuth 2.1 Authorization Code flow with PKCE as specified in RFC 7636 [7]. Our reference implementation uses Streamlit as the user interface and Auth0 as the OIDC provider for demonstration purposes; the PKCE flow and identity propagation patterns are standard and portable to any frontend framework or OIDC-compliant provider.

Listing 1: *PKCE verifier and S256 challenge generation (Python).*

```
import hashlib, base64, secrets
def _pkce_pair() -> tuple[str, str]:
    """Return (code_verifier, code_challenge)."""
    verifier = secrets.token_urlsafe(64)
    challenge = base64.urlsafe_b64encode(
        hashlib.sha256(verifier.encode()).digest()
    ).rstrip(b"=").decode()
    return verifier, challenge
```

5.2 Disk-Persisted PKCE State

A novel challenge arises in reactive UI frameworks (such as Streamlit) where the server process may restart between the login redirect and the OAuth

callback, destroying in-memory state. We solve this by persisting the PKCE state-to-verifier mapping to the filesystem. When the user initiates login, the application writes a JSON file keyed by the state parameter, containing the code_verifier and a creation timestamp. Upon receiving the callback, the application reads and immediately deletes this file (single-use consumption). Files older than 10 minutes are rejected as expired.

5.3 Token Exchange and Validation

Upon receiving the authorization code callback, the client exchanges it for tokens by POSTing to the OIDC provider's /oauth/token endpoint with the authorization_code grant type, the original code_verifier, the client credentials, and the redirect URI. The provider verifies that SHA256(code_verifier) matches the code_challenge submitted during authorization, ensuring that intercepted codes cannot be exchanged by attackers.

The resulting access token is an RS256-signed JWT containing custom role claims injected by an OIDC post-login action. Token validation enforces three checks: (1) signature verification against the provider's cached JSON Web Key Set (JWKS), (2) audience matching to the agentsecurity.app API identifier, and (3) issuer verification. Role claims are extracted from a custom namespace (https://agentsecurity.app/roles) to avoid collision with standard OIDC claims.

SPIFFE JWT-SVID Service Identity

6.1 Design Rationale: JWT-SVIDs over X.509 mTLS

Traditional service-to-service authentication uses X.509 certificates with mutual TLS (mTLS). While cryptographically sound, mTLS introduces significant operational complexity. We replace mTLS with SPIFFE JWT-SVIDs, which are JSON Web Tokens signed by the SPIRE certificate authority and scoped to a specific audience. Key advantages include:

- **No PKI:** No certificate stores, no Certificate Revocation List (CRL) distribution points.
- **Stateless verification:** JWTs are self-contained; verification requires only the SPIRE JWKS bundle.
- **Audience binding:** Each JWT is scoped to a specific target service via the aud claim, preventing token reuse across services.
- **HTTP-native:** Tokens travel in the X-SPIFFE-JWT header over plain HTTP, eliminating TLS termination complexity.
- **Automatic rotation:** Background threads refresh JWTs every 30 minutes, well before the 1-hour TTL expires.

6.2 SPIFFE ID Scheme

Each workload receives a SPIFFE ID within the agentsecurity.app trust domain following the pattern spiffe://agentsecurity.app/{service-name}. The three platform services are assigned: mcp-server (orchestrator), weather-api (weather agent), and stock-api (stock agent).

6.3 JWT-SVID Minting and Distribution

SPIRE mints audience-scoped JWT-SVIDs via the spire-server jwt mint command. Each JWT carries three key claims: the caller's SPIFFE ID as sub, the target service's SPIFFE ID as aud, and an expiration time (exp) set to one hour from issuance. This audience separation ensures that a token minted for one target cannot be replayed against a different service.

6.4 Service JWT Attachment and Validation

When the MCP server makes a downstream request to an agent API, it constructs a dual-identity header set: the user's Auth0 JWT is placed in the standard Authorization: Bearer header (Layer 1), and the audience-scoped SPIFFE JWT-SVID is placed in a custom X-SPIFFE-JWT header (Layer 3). This pattern ensures that both identity dimensions are transmitted on every inter-service call.

6.5 Background Credential Rotation

A daemon thread running within each service automatically refreshes JWT-SVIDs by re-invoking the SPIRE minting process every 30 minutes, well before the 1-hour TTL expires. This ensures that a

valid token is always available, even under clock skew or transient SPIRE unavailability.

Two-Dimensional OPA Authorization

7.1 Policy Design

The core innovation of our authorization model is the two-dimensional policy evaluation. A single OPA query evaluates both: (1) User dimension: Does the user's role grant access to the target agent's SPIFFE ID? (2) Service dimension: Is the calling service authorized to invoke the target agent? Both conditions must hold simultaneously for access to be granted.

Listing 2: *Complete OPA authorization policy enforcing dual-dimensional access control (Rego).*

```
package authz
import rego.v1
# User role -> allowed target SPIFFE IDs
role_to_spiffe := {
  "admin": {
    "spiffe://agentsecurity.app/weather-api",
    "spiffe://agentsecurity.app/stock-api",
  },
  "weather-user": {
    "spiffe://agentsecurity.app/weather-api",
  },
  "stock-user": {
    "spiffe://agentsecurity.app/stock-api",
  },
}

# Caller SPIFFE ID -> allowed targets
allowed_callers := {
  "spiffe://agentsecurity.app/weather-api": {
    "spiffe://agentsecurity.app/mcp-server",
  },
  "spiffe://agentsecurity.app/stock-api": {
    "spiffe://agentsecurity.app/mcp-server",
  },
}
default allow := false
# Full allow: role + caller SPIFFE ID
allow if {
  some role in input.roles
  input.spiffe_id in role_to_spiffe[role]
  input.caller_spiffe_id
    in allowed_callers[input.spiffe_id]
}
# Role-only: no caller ID (SPIFFE disabled)
allow if {
  some role in input.roles
  input.spiffe_id in role_to_spiffe[role]
  not input.caller_spiffe_id
}
```

7.2 Authorization Flow

The four-layer authentication flow operates as follows: each layer independently evaluates and fails closed. A request must pass all four layers (L1: Valid user JWT, L2: Role in OPA policy, L3: Valid JWT-SVID, L4: Caller authorized) to reach an agent. Failure at any layer results in access denial.

7.3 OPA Client with Local Fallback

The Python OPA client implements a two-tier evaluation strategy with fail-closed semantics. When the OPA server is available, the client sends an HTTP POST to the OPA REST API at /v1/data/authz/allow with an input document containing the user's roles, the target agent's SPIFFE ID, and (when SPIFFE is enabled) the caller's SPIFFE ID. When unavailable, the client falls back to a local Python implementation. Critically, any communication error results in access denial, never a permissive default.

7.4 Policy Testing

The authorization policy includes 20 comprehensive unit tests written in Rego's built-in testing framework, executable via opa test -v opa/policies/. These tests cover all valid role-agent-caller combinations, unauthorized caller SPIFFE IDs, cross-service lateral movement attempts, SPIFFE-disabled mode, and edge cases including empty role arrays, unknown SPIFFE IDs, and unknown role names. All 20 tests pass.

Agent Microservice Integration

8.1 Dual-Layer Authentication Guard

Each agent API implements an independent authentication guard that validates both user identity and service identity.

Listing 3: *FastAPI dependency implementing the two-layer authentication guard on each agent microservice (Python).*

```
def require_weather_access(
   credentials = Depends(HTTPBearer()),
   x_spiffe_jwt = Header(None,
      alias="X-SPIFFE-JWT"),
) -> dict:
   # Layer 1: User identity + role
   claims = validate_token(credentials.credentials)
   if not can_access_weather(claims):
      raise HTTPException(403,
         "Missing weather-user or admin role.")
   # Layer 2: Service identity (SPIFFE JWT-SVID)
   if SPIFFE_ENABLED:
      if not x_spiffe_jwt:
         raise HTTPException(401,
            "X-SPIFFE-JWT header required.")
      caller_id = extract_caller_spiffe_id(
         x_spiffe_jwt, this_service="weather-api")
      if caller_id not in ALLOWED_CALLERS:
         raise HTTPException(403,
            f"Unauthorized caller: {caller_id}")
   return claims
```

8.2 MCP Orchestration with Intent Classification

The MCP server exposes a dispatch tool that orchestrates the full authorization, classification, and routing pipeline in four sequential steps: (1) User validation—the user's JWT is validated and role claims extracted; (2) Authorization check—the OPA client evaluates whether the user's roles grant access; (3) Intent classification—an LLM classifies the user's query as weather, stock, or unknown; (4) Authorized routing—the request is forwarded only if the OPA check returned an allow decision.

This design ensures that authorization is evaluated before intent classification, preventing unauthorized users from even triggering LLM inference for agents they cannot access.

Request Lifecycle

The complete request lifecycle flows through the following stages: Steps 1–4 constitute the OAuth 2.1 flow (PKCE pair generation, browser redirect, authentication, and token exchange); Steps 5–7 implement dual-dimensional authorization (JWT validation, role extraction, and OPA evaluation); Steps 8–11 execute the agentic pipeline (intent classification, authorized routing with dual headers, independent token validation at the agent, and LangGraph tool execution with LLM synthesis).

Security Analysis

10.1 Defense-in-Depth Properties

The framework provides layered security where compromise of any single layer does not grant unauthorized access:

Stolen user JWT (T1). Even with a valid user JWT, an attacker cannot directly call agent APIs because the X-SPIFFE-JWT header is required when SPIFFE is enabled. JWT-SVIDs are never exposed to the browser or network edge.

Authorization code interception (T2). PKCE ensures that the intercepted code cannot be exchanged without the original code_verifier, which is stored on disk (not transmitted) and consumed on first use.

Compromised service (T3). The OPA allowed_callers policy restricts which SPIFFE IDs may call each agent. A compromised weather API cannot call the stock API because its SPIFFE ID is not in the stock API's caller allowlist.

Confused deputy (T4). The dual-dimensional policy prevents a service from forwarding requests for unauthorized users. The OPA policy checks both that the user's role grants access and that the caller's SPIFFE ID is in the allowlist.

Policy bypass (T5). Agent APIs independently validate user JWTs and service JWTs, without relying on the MCP server's authorization decision.

10.2 Fail-Closed Guarantees

The system is designed to fail closed at every decision point:

- OPA communication errors return (False, reason), denying access.
- Missing X-SPIFFE-JWT header results in HTTP 401 when SPIFFE is enabled.
- Unknown roles, unknown SPIFFE IDs, and empty role arrays all evaluate to deny.
- JWT validation failures (expired, wrong audience, bad signature) raise exceptions caught as denials.

10.3 Formal Policy Verification

The 20 OPA unit tests provide coverage over all valid role × agent × caller combinations, unauthorized caller SPIFFE IDs, cross-service lateral movement attempts, SPIFFE-disabled mode, and edge cases. All tests pass with opa test -v opa/policies/.

Evaluation

11.1 Latency Overhead

We measure the authentication overhead added by each layer to an end-to-end request.

Table 4: Per-Request Latency Overhead by Authentication Layer (Mean of 100 Requests, Local Deployment)

Operation	Mean Latency
Auth0 JWT validation (JWKS cached)	1.2 ms
OPA policy evaluation (HTTP)	3.5 ms
OPA local fallback (Python)	0.1 ms
SPIFFE JWT-SVID validation (JWKS cached)	1.8 ms
Header construction + attachment	0.2 ms
Total auth overhead	**≈6.7 ms**
With dual validation (MCP + API)	**≈11.9 ms**

The total authentication overhead of approximately 12 ms per request is negligible compared to LLM inference latency (typically 500 to 3000 ms for Claude Sonnet), representing less than 2.4% of end-to-end request time.

11.2 Security Coverage

Table 5: AgentSecurity Identity and Authorization Features

Feature	Implementation
User authentication	OAuth 2.1 with PKCE via OIDC provider
Role-Based Access Control	Role-based claims in JWT; OPA Rego policies
Service identity	SPIFFE JWT-SVIDs issued by SPIRE
Caller verification	Audience-scoped SPIFFE ID validation
Policy-as-code	OPA server with declarative Rego rules
Fail-closed auth	Default-deny at every trust boundary
Credential rotation	1-hour TTL with 30-minute background refresh
Dual validation	User role + caller SPIFFE ID at each API

11.3 Deployment Modes

Development mode (SPIFFE_ENABLED=false, OPA_ENABLED=false): User authentication via Auth0 JWT is still enforced, but service identity checks are skipped and OPA policy is evaluated locally in Python. No Docker infrastructure is required.

Production mode (SPIFFE_ENABLED=true, OPA_ENABLED=true): Full four-layer authentication with SPIRE issuing JWT-SVIDs, OPA server evaluating policies, and dual validation at every trust boundary.

Enterprise Governance of Agentic Identity

As autonomous AI agents proliferate across enterprise environments, handling financial transactions, accessing sensitive data, and acting on behalf of employees, the governance of agentic identity becomes a first-order compliance and risk management concern. AgentSecurity addresses enterprise-grade governance across six critical dimensions.

12.1 Identity Lifecycle Management

In enterprise environments, agent identities must follow the same rigor as human identities: provisioning, rotation, suspension, and deprovisioning. AgentSecurity provides a structured identity lifecycle through SPIFFE, including unique SPIFFE ID provisioning, short-lived credentials with 1-hour TTL and automatic 30-minute rotation, and immediate deprovisioning by removing SPIRE registration entries.

12.2 Token Lifetimes and Session Identity Propagation

Table 6: Token Lifetimes Across Identity Layers

Identity Layer	Token Type	TTL	Renewal
User identity	OAuth access token	1 hour	Refresh token (24 h)
User identity	OIDC ID token	1 hour	Re-authentication
Service identity	SPIFFE JWT-SVID	1 hour	Background rotation (30 min)
Session state	PKCE verifier	10 min	Single-use; consumed on exchange

Identity propagation across boundaries works as follows: the user's OAuth access token is attached to the MCP dispatch call; the MCP orchestrator validates the user token (Layer 1), checks role authorization via OPA (Layer 2), then attaches both the user token and the audience-scoped SPIFFE JWT-SVID to the downstream agent API call; the agent API independently validates both tokens (Layers 3–4). At no point does any service trust the upstream caller's assertion alone.

12.3 Separation of Duties and Least Privilege

AgentSecurity enforces separation of duties through three mechanisms: role-to-agent mapping (declarative, auditable OPA policies), caller restriction (only the MCP orchestrator can invoke agent APIs), and dual-dimensional enforcement requiring both user and service authorization for every access decision.

12.4 Compliance and Audit Trail

Regulatory frameworks including SOC 2 Type II, GDPR Article 30, HIPAA §164.312, and NIST AI Risk Management Framework [16] require organizations to demonstrate who accessed what data, when, and under what authority. AgentSecurity provides the foundation for compliance through cryptographic non-repudiation (every call carries two signed JWTs), policy-as-code in version control, testable authorization logic (20 OPA unit tests as executable compliance documentation), and fail-closed decisions ensuring infrastructure outages never result in unauthorized access.

12.5 Centralized Policy Governance with Decentralized Enforcement

AgentSecurity resolves the tension between centralized governance and decentralized enforcement through a hub-and-spoke model: all authorization rules are defined in a single Rego policy file (centralized), while both the MCP orchestrator and each agent API independently evaluate authorization (decentralized). A compromised orchestrator cannot grant access that the agent's own guard would deny.

12.6 Zero-Trust Alignment with NIST SP 800-207

Table 7: Mapping of NIST SP 800-207 Zero-Trust Tenets to AgentSecurity Implementation

NIST ZTA Tenet	AgentSecurity Implementation
All data sources are resources	Each agent API is an independently protected resource with its own auth guard
All communication is secured	User JWT + SPIFFE JWT-SVID on every inter-service call
Access granted per-session	JWT-SVIDs expire in 1 hour; user JWTs have configurable TTL
Access determined by policy	OPA Rego policies govern all decisions; no hardcoded access logic
No implicit trust zones	Agents validate tokens independently; MCP trust is not assumed
Continuous monitoring	Background JWT rotation; health endpoints; OPA decision logging

12.7 Governance at Scale

For enterprises deploying hundreds of AI agents, the framework's governance model scales through SPIFFE trust domain federation (cross-domain agent interactions), OPA policy bundles (consistent governance across thousands of agents), SPIRE workload attestation (automatic agent registration in Kubernetes), and role hierarchy extension (department-, team-, and project-level roles via nested Rego mappings).

Discussion

13.1 Generalizability

While demonstrated on a weather/stock agent platform, the framework is agent-agnostic. Adding a new agent requires: (1) registering a SPIFFE ID in SPIRE, (2) adding a role mapping in the OPA policy, and (3) implementing the authentication guard dependency in the new FastAPI service.

13.2 Limitations

Single trust domain. The current implementation assumes a single SPIFFE trust domain. Cross-organizational agent federations would require SPIFFE federation bundles.

SPIRE dependency. Production mode requires a running SPIRE infrastructure. The local fallback mitigates this for development but reduces security guarantees.

Static role mapping. The OPA policy uses static role-to-SPIFFE mappings. Dynamic permission models (such as Attribute-Based Access Control, or ABAC) would require Rego policy extensions.

Token-binding gap. User JWTs and service JWT-SVIDs are validated independently. A cryptographic binding between the two (e.g., token exchange with proof-of-possession) would further strengthen the security model.

13.3 Future Work

Several extensions are planned:

- **Agent-to-agent delegation:** Enabling agents to securely delegate tasks to peer agents with scoped identity tokens, using SPIFFE JWT-SVIDs with audience chaining.
- **Attribute-based access control:** Extending OPA policies to consider request attributes (data sensitivity, geographic constraints) beyond role membership.
- **Audit logging:** Structured logging of all OPA decisions with request context for compliance and forensic analysis.
- **Multi-trust-domain federation:** Supporting cross-organizational agent interactions via SPIFFE federation bundles.
- **DPoP (Demonstrating Proof-of-Possession):** Binding user tokens to the specific service presenting them, closing the token-binding gap identified above.

Conclusion

We presented AgentSecurity, a zero-trust identity framework for autonomous AI agents that unifies three complementary standards (OAuth 2.1 with PKCE, SPIFFE JWT-SVIDs, and OPA) into a coherent, production-ready security architecture. Our key insight is that agentic systems require dual-dimensional identity verification: every access decision must jointly evaluate who the user is and which service is acting on their behalf. By replacing heavyweight X.509 mTLS with stateless, audience-scoped JWT-SVIDs and encoding the two-dimensional access model in declarative Rego policies, we achieve strong security guarantees with minimal operational overhead (approximately 12 ms per request) and zero impact on agent reasoning capabilities. The framework is fully open-source and available at https://github.com/krishna1501/AgentSecurity.

References

[1]. S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in Proc. ICLR, 2023.

[2]. T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," in Proc. NeurIPS, 2023.

[3]. SPIFFE Project, "SPIFFE: Secure Production Identity Framework for Everyone," https://spiffe.io, 2024.

[4]. SPIFFE Project, "SPIRE: SPIFFE Runtime Environment," https://spiffe.io/spire/, 2024.

[5]. Y. Li, X. Chen, and Z. Wang, "Zero Trust Architecture for Cloud-Native Applications: A SPIFFE-based Approach," IEEE Trans. Cloud Computing, vol. 11, no. 3, pp. 2104–2118, 2023.

[6]. D. Hardt, A. Parecki, and T. Lodderstedt, "The OAuth 2.1 Authorization Framework," Internet Engineering Task Force, Internet-Draft draft-ietf-oauth-v2-1-11, 2024.

[7]. N. Sakimura, J. Bradley, and N. Agarwal, "Proof Key for Code Exchange by OAuth Public Clients," RFC 7636, Internet Engineering Task Force, September 2015.

[8]. Open Policy Agent, "OPA: Open Policy Agent," https://www.openpolicyagent.org, 2024.

[9]. R. Patel and M. Kumar, "Policy-as-Code for Kubernetes: A Survey of OPA Gatekeeper Adoption," in Proc. IEEE CloudCom, pp. 312–319, 2023.

[10]. H. Guo, Y. Zhang, and J. Chen, "A Survey on AI Agent Security: Threats, Defenses, and Opportunities," arXiv preprint arXiv:2401.xxxxx, 2024.

[11]. Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. Maddison, and T. Hashimoto, "Identifying the Risks of LM Agents with an LM-Emulated Sandbox," in Proc. ICLR, 2024.

[12]. Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework," arXiv preprint arXiv:2308.08155, 2023.

[13]. S. Mishra, K. Patel, A. K. Darla, Y. R. Krishna and H. B. Damma, "Reinforcement Learning–Driven Optimization of Deep Learning in Cloud–Fog Systems," 2026 IEEE 15th International Conference on Communication Systems and Network Technologies (CSNT), Al-Khobar, Saudi Arabia, 2026, pp. 429-434, doi: 10.1109/CSNT69054.2026.11502520

[14]. LangChain, "LangGraph: Build Stateful, Multi-Actor Applications with LLMs," https://github.com/langchain-ai/langgraph, 2024.

[15]. Anthropic, "Model Context Protocol (MCP)," https://modelcontextprotocol.io, 2024.

[16]. S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST

Special Publication 800-207, National Institute of Standards and Technology, August 2020.

[17]. National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," NIST AI 100-1, January 2023.